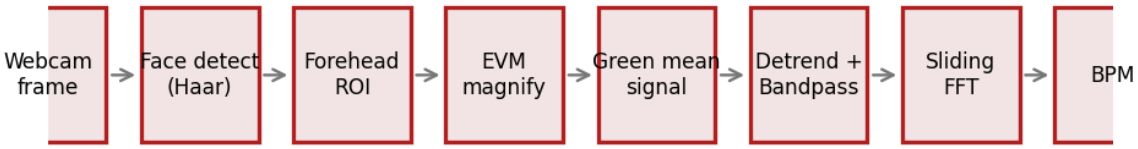
Contactless Heart-Rate Measurement

How a webcam can read your pulse

A DSP walkthrough of remote photoplethysmography (rPPG) with Eulerian Video Magnification



The end-to-end pipeline, from a raw webcam frame to a BPM number.

1. The big idea

Every time your heart beats, it pushes a pulse of blood through the arteries in your skin. Blood absorbs green light more strongly than red or blue, so with each beat the skin gets very slightly greener and then recovers. The change is far too small for the eye to notice — a fraction of one percent of brightness — but a camera records it in the pixel values.

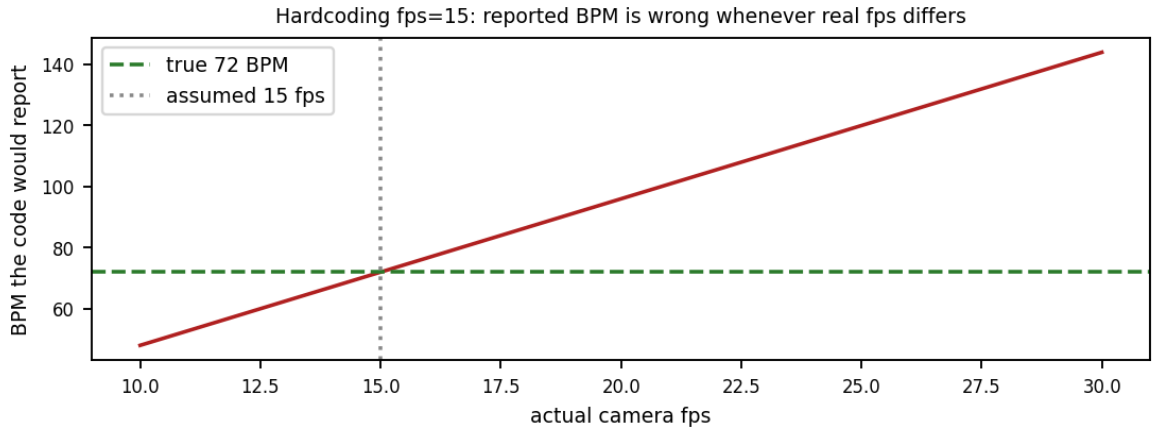
Photoplethysmography (PPG) is the general name for measuring blood volume from light. A pulse-oximeter clip on your finger does it with a dedicated LED and sensor. **Remote PPG (rPPG)** does the same thing at a distance using an ordinary camera — no contact, no hardware. Our job is pure signal processing: dig that tiny periodic colour change out of a noisy video and measure its frequency. That frequency, in beats per minute, is the heart rate.

The whole project is one sentence of DSP: *take a weak periodic signal buried in noise and drift, clean it, and find its dominant frequency.* Everything below is machinery for doing that reliably.

2. Capture — and why frame rate is everything

We grab frames from the webcam with OpenCV. The crucial DSP fact is that the video is a **sampled signal**: each frame is one sample in time, and the **sampling rate is the frames-per-second (fps)**. Every frequency we later compute is measured in cycles per *sample*, and we can only convert that to real Hz (and then BPM) if we know the true fps.

Webcams do *not* deliver a fixed fps. The rate drifts with lighting, USB load, and how long face detection takes each frame. The public reference implementation hardcodes **fps = 15** and builds its frequency axis from that. If the camera actually runs at 30 fps, every reported BPM is doubled; at 10 fps it reads two-thirds of the truth. The error is silent — the number still looks plausible.



A fixed assumed fps scales the answer linearly with the real fps. This is the single most important correctness point in rPPG.

Our implementation instead **measures** the achieved fps at runtime from frame timestamps and uses that measured value everywhere a frequency axis is built:

```
span = stamp_buffer[-1] - stamp_buffer[0]
fps_est = (len(stamp_buffer) - 1) / span
```

3. Finding the skin: face detection and the ROI

We only want skin pixels, so first we locate the face with an OpenCV **Haar cascade** — a classic, lightweight detector that slides simple light/dark rectangle patterns over the greyscale image and reports boxes that look face-like. It is fast on a CPU and needs no deep-learning framework, which keeps the project easy to explain and defend.

We then crop only the **forehead band** — the upper third of the face box, central 60% — rather than the whole face. The forehead is flat, evenly lit, and mostly skin. Including the eyes, mouth, and hair edges would inject motion noise (blinking, talking) that competes with the pulse. Restricting the region of interest (ROI) is a simple, powerful way to raise signal-to-noise before any filtering happens.

4. Eulerian Video Magnification (EVM)

EVM, from Wu et al. at MIT (2012), makes the invisible colour change visible by amplifying it. The name 'Eulerian' means we watch fixed pixels over time (like standing on a riverbank watching water pass) rather than tracking moving objects. The recipe:

(a) Spatial decomposition. Build a **Gaussian pyramid** of each ROI frame — repeatedly blur-and-shrink it (cv2.pyrDown). The pulse is a broad, low-spatial-detail wash of colour across the skin, so it survives on a small, downsampled level while fine texture and noise are discarded. Working on the small level also makes everything cheap.

(b) Temporal filtering. Take one pixel of that downsampled level and look at its value *across all the frames in the window* — that is a time series. Band-pass it to the heart-rate range so only pulse-frequency variation remains. We do this for every pixel and channel at once along the time axis.

(c) Amplification. Multiply that filtered variation by a factor (we use ~40). **(d) Reconstruction.** Upsample it back and add it to the original ROI. The result is a video where the colour pulsation is dozens of times stronger, which makes the downstream signal far easier to measure. In our code this is magnify_roi_stack(), and it is toggleable with --no-evm so you can compare with and without.

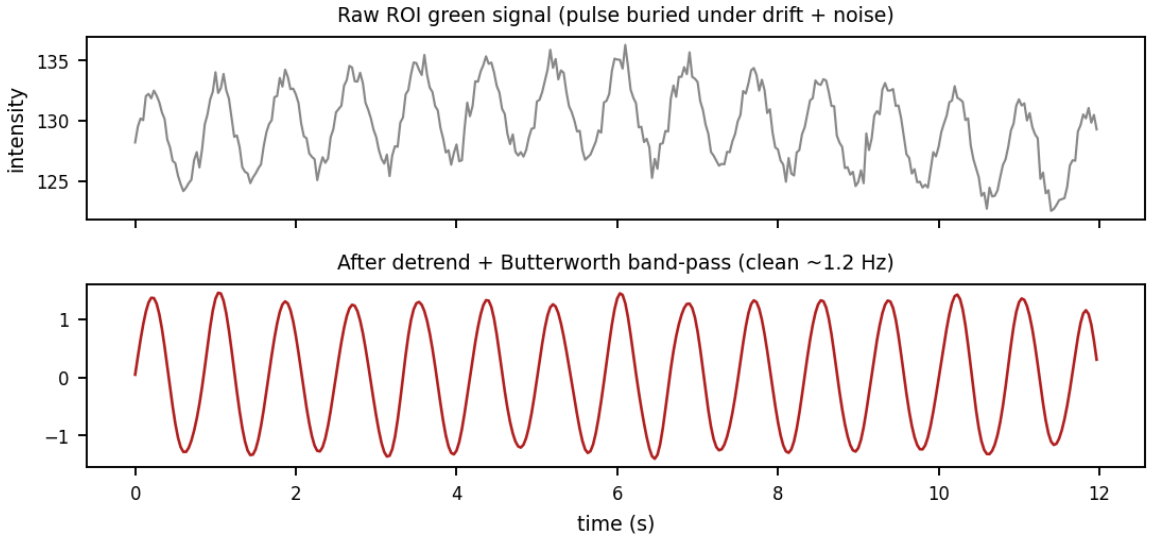
5. From video to a 1-D signal, then cleaning it

For each (magnified) ROI frame we average the **green channel** over all its pixels, giving one number per frame. Stack those over time and we have a 1-D waveform — the raw rPPG signal. Averaging thousands of pixels also averages away random per-pixel noise.

This raw trace is dominated by **slow drift** (lighting shifts, tiny head movements) that is much larger than the pulse. Two DSP steps fix it:

Detrending (scipy.signal.detrend) subtracts the slow-moving baseline so the pulse sits around zero. We then normalise by the standard deviation so the amplitude is comparable run to run.

Band-pass filtering. We apply a **4th-order Butterworth** band-pass at **0.75–3.5 Hz** (that is 45–210 BPM — wide enough to cover a racing pulse, not just a narrow resting range). Butterworth is chosen for its maximally flat passband. Crucially we apply it with filtfilt, which runs the filter forwards then backwards so it introduces **zero phase shift** — the pulse peaks stay where they belong in time.

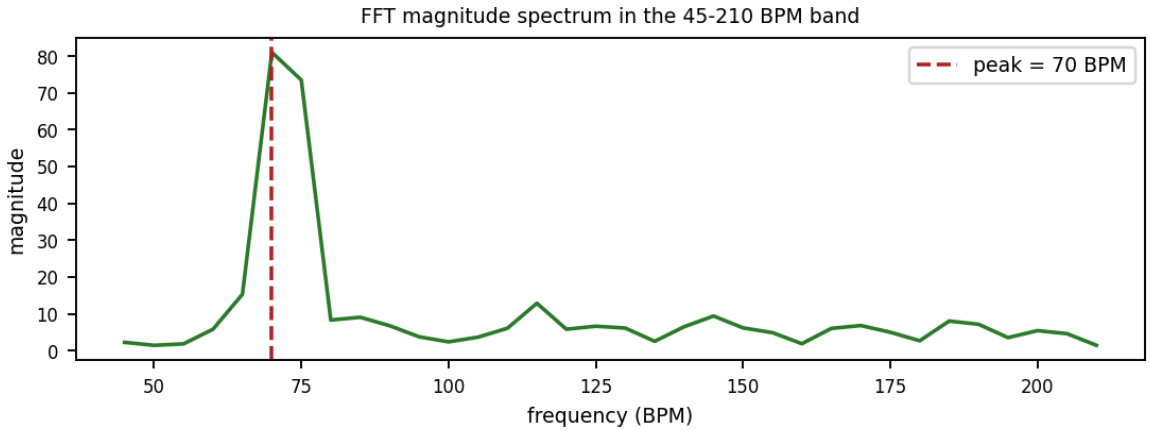


Top: the raw signal — the pulse is invisible under drift and noise. Bottom: after detrending and the zero-phase band-pass, a clean periodic wave emerges.

6. Measuring the rate: the FFT

A clean periodic wave still has to be turned into a number. We take the **Fast Fourier Transform (FFT)** of the filtered signal over a sliding ~12-second window. The FFT decomposes the wave into its constituent frequencies and tells us how strong each one is — the **spectrum**. A steady heartbeat shows up as a tall peak at its frequency.

Before the FFT we multiply by a **Hanning window** to taper the ends of the segment to zero. Without it, the abrupt cut at the window edges smears energy across the spectrum ('spectral leakage') and blurs the peak. We then find the tallest peak *within the 0.75–3.5 Hz band* and read off its frequency.



The spectrum of the cleaned signal. The dominant in-band peak is the heart rate; here it sits at 72 BPM.

Finally the conversion is trivial: **BPM = peak frequency in Hz × 60**. The window slides forward and the estimate refreshes about once a second, so you get a continuously updating heart rate.

```
bpm, freqs, spectrum, peak = dominant_bpm(filtered, fps_est)
# inside: freqs = np.fft.rfftfreq(n, d=1/fs); bpm = freqs[peak]*60
```

7. The four classic mistakes (and our fixes)

Pitfall	Why it breaks	Our fix
Hardcoded fps	Frequency axis wrong → every BPM wrong	Measure fps from timestamps
Narrow 60–120 band	Fails silently for high/low rates	Wide 0.75–3.5 Hz band
Whole-face ROI	Eyes/mouth motion drowns the pulse	Forehead-only ROI
No detrending	Lighting drift dominates the FFT	Explicit detrend before band-pass

8. Where each idea lives in the code

Concept	Function in heart_rate.py
Measured frame rate	`the fps_est computation in main()`
Forehead ROI crop	`forehead_roi()`
Gaussian pyramid up/down	`gaussian_downsample / gaussian_upsample`
EVM magnification	`magnify_roi_stack()`
Detrend + Butterworth	`bandpass_trace()`
FFT peak → BPM	`dominant_bpm()`
Live plots	`LivePlots class`

9. Try it and prove it to yourself

Run the live estimator, sit still in even light, and after ~6–12 s compare the on-screen BPM against your pulse counted by hand (count 30 s, double it). They should agree within roughly ±5–10 BPM.

```
pip install -r requirements.txt
python heart_rate.py # live webcam, EVM on
python heart_rate.py --no-evm # compare without magnification
python test_signal_chain.py # prove the DSP on synthetic pulses
```

The self-test is worth understanding: it manufactures a noisy sine wave at a known BPM (with fake lighting drift) and confirms the pipeline recovers that exact rate at several frame rates. It isolates the DSP from the camera, so you can trust the maths before you ever trust the webcam.

10. Honest limitations

rPPG is genuinely fragile. Motion, uneven or flickering light, and darker skin tones all reduce the already-tiny signal. Because we report the strongest in-band frequency, under bad conditions the system will still output a confident-looking number that is really just the loudest noise. It is a wonderful demonstration of sampling, filtering, and the FFT — not a medical device. Understanding *why* it can be fooled is as much the lesson as watching it work.

Reference: H.-Y. Wu, M. Rubinstein, E. Shih, J. Guttag, F. Durand, W. Freeman, “Eulerian Video Magnification for Revealing Subtle Changes in the World,” ACM TOG (SIGGRAPH) 2012. Project: people.csail.mit.edu/mrub/evm/